



Exploring Federated Learning on Battery-powered Devices

Zichen Xu
Nanchang University
xuz@ncu.edu.cn

Li Li^{*}
Shenzhen Institutes of Advanced
Technology, Chinese Academy of
Sciences
li.li@siat.ac.cn

Wenting Zou
Nanchang University
wentingzou@email.ncu.edu.cn

1 ABSTRACT

Smartphones generate private data ubiquitously that serves Big data analysis. Without violating privacy issue, analytic companies want to understand and learn features from these data, which fits the nature of federated learning. Federated learning invites multiple participants to train a learning model (e.g., Artificial Neural Network) while guaranteeing the data privacy. However, processing federated learning is heavy on batteries, which is against the ubiquity feature of smartphone. In this paper, we explore the possibility of enabling federated learning on many battery-powered devices. We share our observations on supporting federated learning using battery power, and propose a two-layered strategy to process the learning on batteries with a reasonable tradeoff. The first layer improves the initialization of the federated learning while the second layer explores local energy saving potential. The results show that our work can finish the learning process successfully, consuming 20% less energy on average, and pays negligible overhead (average 0.1%) and accuracy loss (2% at most), as compared to the default setting.

2 INTRODUCTION

Smartphone devices, powered by batteries, outreach connection between users and their environment. Equipped with many sensors, smartphones can collect rich data around user, ubiquitously. These data are valuable resources for Big data analytic companies as they want to understand user behaviors and their networks [4]. Acquiring these data raises a big question mark on preserving users' privacy with any

services provided. For example, the scandal swirling around the Facebook App and Cambridge Analytica has begun to usher in a new era for this once-ignored community of privacy researchers and developers. Before understanding and learning ubiquitous user data, the privacy is the concern.

Federated learning is a framework to process a shared model training on the data location itself. Unlike standard approach [11], federated learning does not require the training data to be centralized in data center [13], instead, it only processes meta data for training convergence in a centralized location. Thus, federated learning is a sufficient match for data analysis on the cloud-edge infrastructure, concerning data privacy. This learning technique allows the users to collaboratively build a shared learning model while preserving all training data on their own devices [2]. In particular, a mobile device computes the updates to the current global model on its local training data, which is then aggregated and fed-back by a central server, so that all the participants have access to the same global model in order to compute their new updates. This process is repeated until an accuracy level of the learning model is reached [1].

When operating in a slow and unstable network, federated learning can be costly. The whole learning process requires many rounds of communication between the central server and smartphones before the modeling training is converged. Former researches on federated learning usually focus on the modeling process itself, and simply assume that the phone is training data while in-charging. This strong assumption contradicts the original purpose of learning data, that is understanding data collected by smartphones ubiquitously. To make federated learning practical, we have to make it running on batteries [6].

To seize the potential of federated learning on ubiquitous data and preserve local data privacy from smartphones, we empirically study the energy behavior and profiles of federated learning on smartphones. In a federated learning system, a mobile device completes the local training process and sends the gradient update. The local training data and the current battery level of a specific mobile device can highly impact the training performance. This impact is *not*

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

ACM TURC 2019, May 17–19, 2019, Chengdu, China

© 2019 Association for Computing Machinery.

ACM ISBN 978-1-4503-7158-2/19/05.

<https://doi.org/10.1145/3321408.3323080>

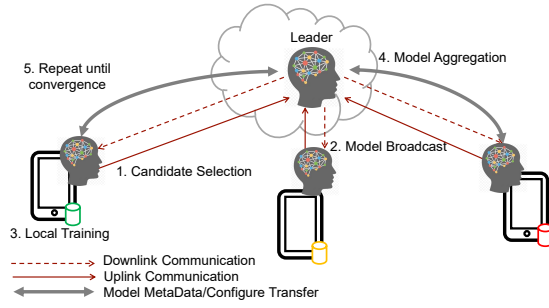


Figure 1: A typical procedure of federated learning in cloud-edge infrastructure.

monotonic between latency [8] and runtime power during processing. This observation shed the light on making the learning jobs possible in an energy constrained environment.

In this paper, we explore the possibility of enabling federated learning on many battery-powered devices. We share our observations on supporting federated learning using battery power, and propose a two-layered strategy to process the learning on batteries with a reasonable tradeoff. The first layer improves the initialization of the federated learning while the second layer explores local energy saving potential. Our early results show that in practice, our work can finish the learning process successfully, consuming 20% less energy on average, and pays negligible overhead (average 0.1%) and accuracy loss (2% at most), as compared to state of the arts. As such, we prove it is possible to retain ubiquity for smartphones on performing federated learning unplugged. Specifically, our contributions are:

- We identified the question on enabling federated learning on battery-powered devices.
- We conducted a series of experiments on understanding the energy use behavior of federated learning on smartphones, and observed the potential of performing learning on batteries.
- We proposed a mechanism based on our observations, making federated learning on battery-powered devices possible.
- We prototyped our work on physical testbed and verified our solution.

3 OBSERVATION AND MOTIVATION

Federated learning (FL) is commonly realized as a networked service between the leader node (i.e., centralized orchestration node) and training clients, supporting the following jobs:

- $\text{variance } v \leftarrow \text{train}(\text{model } m, \text{data } d)$
- $\{\text{model } m, \text{revision } s\} \leftarrow \text{convergence}(\text{variance } v)$

FL sets a global processing broadcast for train processes across all data sites and ensures that subsequent converge calculation will return a smaller variance v after each epoch

of training. In other words, FL recurrent network training in a master-slaver manner.

Figure 1 (red arrow lines) illustrates the framework initialization and (grey arrow lines) depicts train and converge execution. The candidate client (i.e., smartphones with data) provides device specification and meta data when issuing the query. The leader proposes a training model m and target training data d . After each round of training, the leader calculates the model variance and configure the model setup to increment revision model setup s . Specifically, FL operates in Figure 1 as follows:

- (1) The leader heartbeats any connected candidates (red dash arrow) and tries to receive feedback from clients as the candidate (red solid arrow), STEP 1.
- (2) After initializing the setup, leader broadcasts the current global model state (e.g., model parameters) to all candidates, and starts the learning epoch, STEP 2.
- (3) All candidates start to train their local data, based on the broadcast model and configure, STEP 3.
- (4) After each training epoch (i.e., all devices finished their training), the leader aggregates model gradients and applies the update. Based a calculated weighted results average, leader sends the updated model parameters all candidates, STEP 4.
- (5) The whole training iterates until the global model converges under certain thresholds, STEP 5.

During the whole process, all devices have to be synchronized before next step. The leader does not access any local data so that data privacy is well preserved. The whole process clearly several deficient design issues that may require further investigation. Here we empirically studied the whole process on modern smartphone and share our observations.

Observation 1: On-device training is heavy in power consumption and NOT user-friendly.

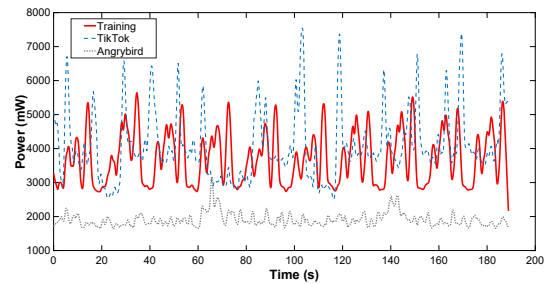


Figure 2: Power snapshot of FL and other baselines (Samsung Galaxy S4).

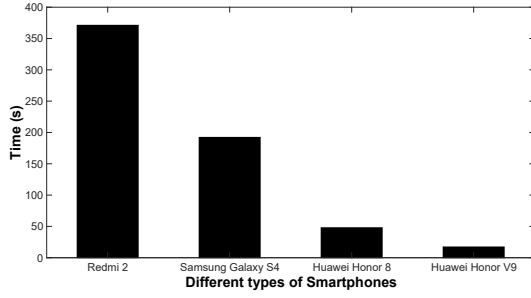


Figure 3: Training completion time differs with various hardware configurations.

We first compare the power consumption profile by comparing FL jobs (i.e., LeNet jobs [5]) and other typical smartphone applications. During the experiment, we used a Monsoon Power Monitor to monitor the runtime power consumption of the whole phone. Figure 2 shows the power consumption of the training process, compared to two popular Android apps (e.g., TikTok, video play and AngryBird, game [9]). The average power of the training process is 4546mW, which is higher than AngryBird (2052 mW), but almost comparable to TikTok (4951 mW). However, FL may not provide the entertainment experience as TikTok for users. To make FL practical and persuade regular users to join the training process, we recommend an incentive mechanism to encourage and recruit feasible participants.

Observation 2: The training completion time in each epoch varies from different devices.

We repeat the above FL jobs on four popular smartphones, i.e., Redmi2, Samsung Galaxy S4, Huawei Honor 8, and Honor V9, respectively, as shown in Figure 3. The completion time of the same training process with the same size of data set, differs on different mobile platforms. At most, Redmi 2 spends 12X more time on training process, as compared to Honor V9. Recall that in each epoch, every phone has to be synchronized before orchestration in the leader node. In our case, all three other phones need to wait until Redmi 2 finishes its job before next step. If no other jobs running, all the rest devices are wasting their valuable CPU time. We urge that *the leader shall select candidate carefully or has a way to coordinate the system so that local training time across all participants is balanced.*

Observation 3: FL jobs preemptively acquire CPU shards that it prevents system entering low-power state.

To explore where the power is consumed by FL jobs, we studied the runtime system stats and find that FL jobs usually force the CPU governor running in the highest frequency, no matter the process is foreground or background, as shown in Figure 4. If we configure the CPU governor and force FL jobs running in different frequency levels [7] (e.g., using Dynamic Voltage and Frequency Scaling, DVFS [3]), we observe a

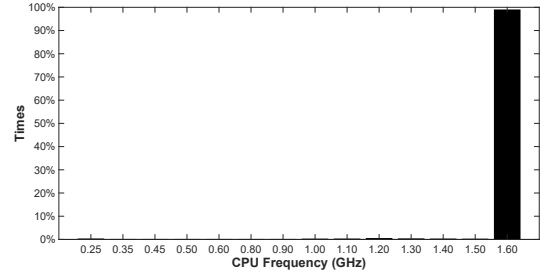


Figure 4: FL jobs always run in the highest frequency.

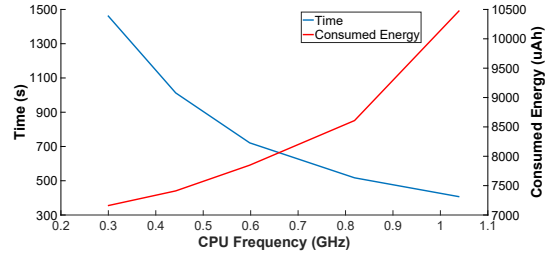


Figure 5: Training completion time.

convex energy curve as we increase the CPU frequency while running FL jobs, illustrated as the red line in Figure 5. For training local data, we may need to run the job as soon as possible. However, in a federated learning framework, as aforementioned in Figure 3, running too fast may cause unnecessary idling waiting. It is obviously better if (1) all participants can finish their local jobs at the same/similar time; (2) Some phones can run FL jobs slower on purpose, for synchronization, and seizing the opportunities for energy saving.

4 SYSTEM DESIGN

In this section, we discuss our modeling process and our two-layered optimization design. The first layer (Step 1-2) helps to select the best candidate, and invites the wanted phone into the learning process. As aforementioned in our observation, federated training is costly, such that either phone user do not want to participate or data/battery provided in the candidate phone is not sufficient to finish the whole process. During the whole process, if one device fails, it may crush the whole process and waste other phones' computation resource and time. In this layer, we provide an incentive mechanism using auction [10]-like strategies to negotiate with bidding. The second layer is the local energy optimization based on predicted deadlines in each training epoch. If the target phone is not the slowest phone, we allow the training job running in low power modes to save energy and also meet the synchronization goal.

Notations	Definition
m	Model type
v	Model variance
n	Number of participants
R	Overall revenue
r	Assigned revenue
d	Number of data objects
b	Remain battery volume
c	Number of cpu cores
f	Core frequency
i	index of participants

Table 1: Symbols and Notations

4.1 Modeling Setup

Federated Learning Network (FLN), illustrated in Figure 1, composes a set of smartphones formed by learning service providers, i.e., the leader. To simplify our discussion and focus on the energy problem, in the rest of the paper we consider all phones are connected in good conditions; however, we underline that the proposed mechanisms can be easily extended to consider other network parameters like bandwidth and others, such as in federated learning modeling [12].

The learning network is managed by a single service operator that provides ubiquitous access to all mobile customers (MCs), while each participant to the trading market is the owner of a battery-powered phone. Phone owners lease the unused capacity of their phone batteries, and available data to-be-trained, so that the service operator can rent the available devices to offload the data training tasks when, for example, this training job requires data provided by the mobile network. Table 1 summarizes all notations.

4.2 Bidding Modeling in Initialization

The whole process starts with dividing the revenue R from finishing a training process. As aforementioned, the whole process may need several rounds of training and communication between edge devices and the inference cloud. The first part starts with selecting a number of candidates for training. Thus, it is easy to see that:

$$R = \sum_{i=0}^{n-1} r_i \quad (1)$$

in which r_i is the assigned revenue for client i . To compute the revenue assigned to each user i , we take a look at how many related objects to be trained (i.e., d), and the remaining capacity of battery (i.e., b) that allowed us to train. Thus,

$$r_i = d * b^\alpha \quad (2)$$

in which α is a weight coefficient ranging from 0 to ∞ . When $\alpha = 0$, our model processes exactly the same as the original federated learning. From the cloud perspective, we want more data and more battery to compute and train the model,

thus, the more data, the higher revenue, and it is the same for the battery. When there is no useful data available in one device, it shall not be selected, thus, the relative revenue is 0. A negotiation processes with all possible candidates to select the training set S , where $S = n$.

4.3 Training Completion Time Model

For a specific training task (e.g., a particular DNN model), the time required to finish an training epoch on different devices is totally different. The training completion time is highly correlated with the following factors. First, since different devices can have different number of data objects to be trained, the training completion time can be determined by the amount of data objects d_i . Second, different clients can have totally different training platforms (e.g., CPU, GPU), thus the training completion time can also depend on different training platforms. Third, for a certain training platform, the hardware configuration can be different (e.g., number of cores, core frequency). Thus, we model the training completion time for one epoch as follows:

$$t_i = \frac{\theta * d_i}{c^\alpha f^\beta} \quad (3)$$

θ represents the coefficient which is different for different training platforms and devices. d_i represents the number of data objects required to be trained in this training epoch. c represents the number of cpu cores adopted during the training process. f represents the core frequency. α and β are the corresponding coefficients that can be obtained through the system identification process. We can find that the more data objects need to be trained, the longer time it requires to complete the training. The more number of cpu cores and high core frequency can help accelerate the training process.

4.4 DVFS Power Model

During the training process, the power consumption (e.g., different devices, different core frequency) can be different. Thus, we model the training power from CPU as follows:

$$P_i = P_{BL, N_c} + \sum_{i=1}^{N_c} P_{core, U_i, f_i} \quad (4)$$

where N_c is the number of cores enabled, P_{BL, N_c} is the baseline CPU power with N_c enabled cores, and P_{core, U_i, f_i} is power increment of core i when it is working at frequency f_i with utilization U_i .

Moreover, the single core power P_{core, U_i, f_i} can be modeled as follows:

$$P_{core, U_i, f_i} = \gamma * f_i * U_i + \gamma_{base} \quad (5)$$

in which, γ is the corresponding power coefficient. At last, the energy required for a specific training epoch on client i

can be modeled as follows:

$$E_i = P_i * t_i \quad (6)$$

4.5 Auction-based Initialization

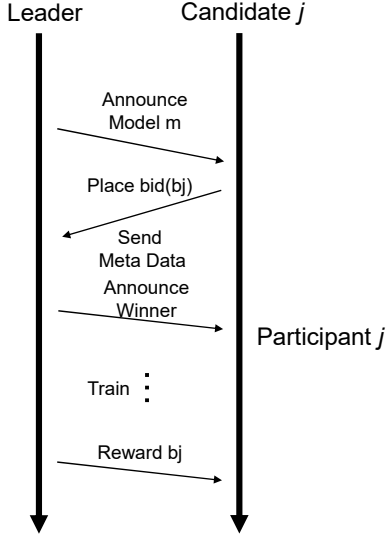


Figure 6: The auction bidding process.

We specialize in a federated learning system with a crowd of mobile candidates. A subset of mobile clients will be recruited to participate the federated learning system. There is a compensate of c_i for each selected mobile client. For mobile user i participating in the learning process (denoted as y_i), we give a quality score to quantify the quality of the services/data that the mobile user may be able to provide to the training task. The quality score is primarily determined by the available data size and the battery can be used for training. It is task-dependent. The leader estimates the quality scores and lists the wanted candidates.

The overall Quality of Learning q quantifies the quality of services/data the crowd of clients is potentially capable of providing, which could be given by a function $g(\cdot)$ that satisfies the following attributes: 1) $g(\cdot)$ is a monotonically non-decreasing function of y_i and x_i , where x_i is a binary value indicating whether mobile user i is recruited or not; and 2) $g(\cdot)$ returns a value in $[0,1]$. The first attribute reflects the nature that the training quality will not deteriorate as the training data (and battery) increases. We believe that the training should be completed with a minimum quality requirement r throughout the training process. The cloud operator recruits mobile clients and ensure that the training quality can meet the requirement.

During the initialization, our incentive mechanism design can be formulated as a reverse or procurement auction mechanism design problem. In this auction, as shown in Figure 6,

it starts with 1) the cloud operator (the leader) announces a federated learning model to mobile candidates (bidders); 2) each candidate i submits a bid b_i ; 3) the cloud operator compares the weighted quality based on bid and score, to select the winners and determine payments; 4) winners carry out the learning task and submit results to the cloud operator; 5) the cloud operator checks the results and makes payments to winners. In the following, we use candidate and bidder interchangeably.

The cloud operator must complete the learning task j to the required quality r . Moreover, the cloud operator wishes to conserve money and minimize its expected expenditure by selectively recruiting mobile clients. Specifically, an auction mechanism takes the bid vector B and quality requirement r as input and returns a winner vector $x = [x_1, \dots, x_2, \dots, x_M]$, where $x_i = 1$ if mobile client i wins, and $x_i = 0$ otherwise; it also returns a payment vector $P = [p_1, \dots, p_i, \dots, p_M]$, where p_i is the payment that the cloud operator will make to mobile user i . Based on the output of the auction, the payoff of mobile user i is defined as $u_i = p_i - c_i$ when $x_i = 1$, $u_i = 0$ when $x_i = 0$. The expenditure of a reverse auction is the sum of the payments $\sum_i^M p_i$ to all mobile users (bidders). Winner selection problem can be formulated as the following Integer Programming (IP) problem:

IP-Winner:

$$\min \sum_{i=1}^M \beta_i(w_i)x_i \quad (7)$$

Subject to:

$$q = g(Z, X) > r \quad (8)$$

where β_i is the weighted cost for candidate i .

4.6 Online Energy Optimization

With the selected clients, we try to optimize the energy consumption locally while guaranteeing the training performance. The online energy optimization contains two main parts: 1) the training deadline prediction and 2) the online DVFS control.

4.6.1 Training Deadline Prediction. For the current federated learning system, we adopt the synchronous model averaging scheme. It means that, in each communication round, each client sends the model updates to the central server. When the server receives the model updates from all the selected clients, the central server performs model averaging and send back the updated model. After that, the training enters the next round. In order to decrease the whole training time, we try to minimize the training time of each communication round. We can find that the training time of each communication round is determined by the client that requires the longest time to finish the training. In order to

make this duration short, we ask that device to increase the number of cores it uses and the core frequency.

In the beginning of each communication round, the clients send the following information $\langle c_i^{max}, f_i^{max}, d_i \rangle$ to the central server. c_i^{max} represents the maximum number of cores that can be used to participate in this round of training. f_i^{max} represents the highest core frequency that can be leveraged for the training process. d_i represents the amount of data objects required to be trained in the current round. We define the training deadline $t^{deadline}$ of the current communication round as follows:

$$t^{deadline} = \text{MAX}\left\{\frac{\theta * d_i}{c_i^{max} \alpha f_i^{max} \beta}\right\}, i \in m \quad (9)$$

The training deadline represents the shortest we need to use in each communication round.

5 RESULTS

We have implemented a prototype of federated learning using our mechanism on six popular smartphones, namely Redmi, Redmi2, Samsung Galaxy S4, Huawei Honor 8, Honor V9 and Nexus 6. The FL workloads are standard LeNet jobs [5] on classifying local image data. To capture the power data, we used a Monsoon Power Monitor to monitor the runtime power consumption of the whole phone.

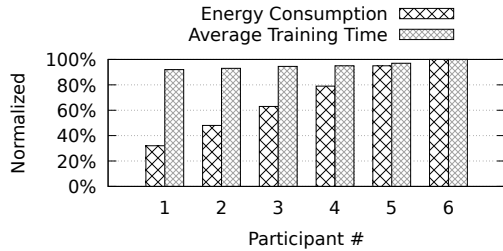


Figure 7: Energy consumption and average training in six phones.

Figure 7 illustrates the overall performance gain using our mechanism. Our prediction model can set a reasonable deadline in each epoch. As illustrated in Figure 7, all phones almost finish at the same time. Based on the deadline, some phones, for example, phone 1, can slow down and save energy. At most, Phone 1 receives 64% more energy savings, compared to the slowest phone. If we consider all phones running in the default governor, the whole framework can achieve approximate 20% energy savings.

To further explore the energy gain breakdown, we show an ordered CPU frequency selection during the whole process, namely participant 4 in Figure 7. This phone spends 70% of its time on the highest frequency (2.7GHz). When it realizes the job can be finished earlier than the designed deadline, it scales to a medium frequency (1.5-2.5GHz), until running in the low-power mode (300MHz). Local DVFS adaption

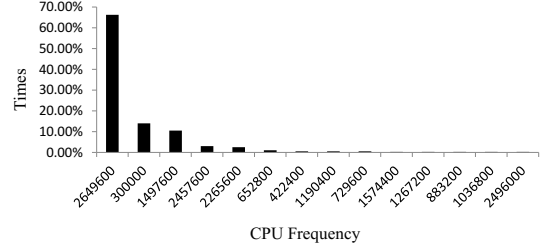


Figure 8: CPU frequency selected by the governor during the training process on Nexus 6.

based on our modeling results help achieve significant energy savings for devices that can finish training before deadline.

6 CONCLUSION AND FUTURE WORK

In this paper, we explore the possibility of enabling federated learning on battery-powered devices and propose a two-layered design to process the learning on batteries with a reasonable tradeoff. The results show that our work can finish the learning process successfully, consuming 20% less energy as compared to the default setup. Immediate future work includes further identifying modeling coefficients and performing tests on more devices.

REFERENCES

- [1] S. Caldas et al. 2018. Federated Kernelized Multi-Task Learning. In *SYSML*.
- [2] H.B. et al. 2017. Communication-Efficient Learning of Deep Networks from Decentralized Data. In *AISTATS*.
- [3] Lee J et al. 2007. Dynamic voltage and frequency scaling (DVFS) scheme for multi-domains power management. In *2007 IEEE Asian Solid-State Circuits Conference*.
- [4] J. Konecny et al. 2016. Federated Learning: Strategies for Improving Communication Efficiency. In *Nips*.
- [5] Jia L and Sun Y. 2018. Digital Recognition Based on Improved LENET Convolution Neural Network. In *Proceedings of the 2018 International Conference on Machine Learning Technologies*.
- [6] Li L et al. 2017. eDelta: Pinpointing Energy Deviations in Smartphone Apps via Comparative Trace Analysis. In *2017 Eighth International Green and Sustainable Computing Conference (IGSC)*.
- [7] Li L et al. 2017. SceneMan: Bridging Mobile Apps with System Energy Manager via Scenario Notification. In *2017 IEEE/ACM International Symposium on Low Power Electronics and Design (ISLPED)*.
- [8] Deng N et al. 2015. Tell-tale tails: Decomposing response times for live internet services. In *2015 Sixth International Green and Sustainable Computing Conference (IGSC)*.
- [9] Yang Q et al. 2019. Mobile Gaming on Personal Computers with Direct Android Emulation. In *The 25th ACM Annual International Conference on Mobile Computing and Networking (MobiCom)*.
- [10] Yang S et al. 2018. On Designing Distributed Auction Mechanisms for Wireless Spectrum Allocation. *IEEE Transactions on Mobile Computing (TMC)*.
- [11] V. Smith et al. 2017. Federated Multi-Task Learning. In *Nips*.
- [12] N. H. Tran et al. 2019. Federated Learning over Wireless Networks: Optimization Model Design and Analysis. In *INFOCOM*.
- [13] Zichen Xu et al. 2016. Blending on-demand and spot instances to lower costs for in-memory storage. In *INFOCOM 2016*. 1–9.